

НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ОДЕСЬКА ПОЛІТЕХНІКА»
МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ІНСТИТУТ КОМП'ЮТЕРНИХ СИСТЕМ
КАФЕДРА ІНФОРМАЦІЙНИХ СИСТЕМ

Протокол лабораторної роботи №1
«розгортання середовища та порівняльний аналіз мовних моделей»
з дисципліни: «Обчислювальні методи»

Виконав(-ла):

студент(-ка) групи AI-244
Ваулін Данило
Святославович

Перевірив:

ас. Баранов О.М.

Завдання 1. Ціна роздумів: міркувальна vs інструктивна модель

Виконано запити до двох моделей через ендпойнт `/api/generate`. Спочатку – просте фактичне питання, потім – задача на міркування (5 запусків для кожної моделі). Перед серією вимірів зроблено прогрівальний запит.

Таблиця 1 – Фактичне питання «Столиця України? Відповідай одним словом.»

Показник	qwen3:4b	llama3.2:3b
response	Київ	Київ
eval_count	246	5
Час, с	19.00	6.07
Токенів/с	26.18	46.15

Таблиця 2 – Задача на міркування (середнє з 5 запусків)

Показник	qwen3:4b	llama3.2:3b
Правильних із 5	5 / 5	2 / 5
eval_count	1203	25
Час, с	49.29	1.5

Типова помилка інструктивної моделі: відповідь «10 грн», отримана без урахування того, що різниця у 100 грн розподіляється між двома доданками.

Відповіді на питання (крок 6)

1. Куди пішли додаткові токени в міркувальній моделі?

Відповідь: Додаткові токени йдуть на внутрішній процес роздумів моделі — аналіз запиту, перевірку гіпотез та побудову логіки перед видачею тексту. Для простих питань це просто зайве навантаження.

2. На яких типах задач міркувальні моделі дають перевагу?

Відповідь: Вони дають перевагу там, де потрібна точна логіка: у математиці, написанні коду та багатоетапній аналітиці. Використовувати їх для форматування, базового перекладу чи простих фактів недоцільно, бо це лише марнує час.

3. Чим відрізняється процес навчання міркувальної моделі від інструктивної?

Відповідь: Головна відмінність у тому, що міркувальні моделі додатково тренують через навчання з підкріпленням. Роботу моделі оцінюють за якість і правильність самого ходу думок, а не тільки за підсумкову відповідь. Це вчить її шукати й виправляти власні помилки в процесі.

4. Чи означає більша кількість токенів, що модель «розумніша»?

Відповідь: Ні, більша кількість токенів показує лише те, що модель проходить додатковий етап аналізу. Для складних задач це підвищує якість, а на простих просто витрачає ресурси без жодного корисної ефекту.

5. Як пояснити різницю у швидкості генерації між моделями?

Відповідь: Усе залежить від архітектури та алгоритмів. Qwen3:4b повільніша, бо перед відповіддю запускає ресурсомісткий процес міркувань, тоді як Llama3.2:3b — це легка інструктивна модель, яка генерує текст одразу без попереднього аналізу.

Висновок за завданням 1

Експеримент показав, що два показники — швидкість генерації та загальний час відповіді — не можна плутати. Перший характеризує обладнання й розмір моделі, другий — обсяг роботи, яку модель собі призначила. На фактичному питанні режим міркування виявився чистою перевитратою: результат однаковий, витрати відрізняються у 3,13 разів. На задачі про ручку й зошит ситуація протилежна — саме розписаний хід дозволив уникнути інтуїтивно привабливої, але неправильної відповіді. Практичний висновок: тип моделі треба обирати під тип задачі, а для агента з 8 послідовними викликами різниця складе 394,3 с проти 12,0 с, що робить міркувальну модель непридатною для налагодження.

Завдання 2. Роль system-повідомлення: керування поведінкою моделі

Таблиця 3 – Вплив system-повідомлення

Модель	System	Довжина відповіді, символів	Мова відповіді	Кількість речень	Інструкцію виконано?
llama3.2:3b	лаконічний асистент	811	Українська + Англійська + дивні символи	10	Ні
llama3.2:3b	українською, 2 речення	179	Українська + дивні символи	2	Частково
qwen3:4b	лаконічний асистент	204	В більшості українська	2	Так
qwen3:4b	українською, 2 речення	137	Українська	1	Так

Таблиця 4: Витрати токенів

Модель	System	prompt_tokens	completion_tokens	total_tokens
llama3.2:3b	лаконічний асистент	52	596	649
llama3.2:3b	українською, 2 речення	59	69	119
qwen3:4b	лаконічний асистент	57	2243	2571
qwen3:4b	українською, 2 речення	64	601	986

Відповіді на питання (крок 6):

1. Яка модель точніше дотрималась обмеження «максимум двома реченнями»?

Відповідь: Qwen3:4b впоралася чудово: при першому системному промті вона видала 2 речення, а при другому — 1 речення. А вот Llama3.2:3b спочатку видала аж 10 речень замість стислої відповіді, а при другій спробі хоч і вклалася у 2 речення, але згенерувала текст із сторонніми символами.

2. Чи були випадки, коли модель відповіла не українською, попри пряму вказівку?
Відповідь: Так, Llama3.2:3b у першому запуску відповіла сумішню української, англійської та дивних символів. Qwen3:4b трималася стабільно: у першому випадку відповіла здебільшого українською, а в другому — повністю українською.
 3. Як змінився prompt_tokens при зміні system-повідомлення і чому?
Відповідь: Кількість вхідних токенів зросла у обох моделей: у Llama з 52 до 59, а у Qwen з 57 до 64. Це сталося через те, що нове системне повідомлення "українською, 2 речення" за обсягом довше за попереднє "лаконічний асистент", тому й вимагає більше токенів для токенизації.
 4. Чому в qwen3:4b значення completion_tokens значно більше, ніж у llama3.2:3b, навіть при однаковій довжині видимої відповіді?
Відповідь: У Qwen показник completion_tokens досягає 2243 та 601 проти 596 та 69 у Llama. Причина в тому, що Qwen3 є міркувальною моделлю, і в completion_tokens враховуються не лише слова видимої відповіді, а й усі токени внутрішнього блоку роздумів (thinking)
- Відповіді на питання (крок 7)
1. Чому моделі по-різному дотримуються прямих інструкцій щодо довжини та мови?
Відповідь: Все залежить від архітектури. Llama3.2:3b — це звичайна мала інструктивна модель, якій важко контролювати точні ліміти генерації. Qwen3:4b перед відповіддю будує ланцюжок міркувань, що дозволяє їй краще спланувати обсяг і мову.
 2. Чим system-повідомлення відрізняється від user-повідомлення з точки зору моделі?
Відповідь: Системні інструкції мають вищий пріоритет для налаштування поведінки завдяки спеціальним рольовим токенам. Проте якщо інструкції конфліктують, модель може зорієнтуватися на user-повідомлення, бо воно знаходиться ближче до кінця вхідного контексту.
 3. Чому міркувальна модель витрачає більше вихідних токенів при однаковій

видимій довжині?

Відповідь: API підраховує загальну кількість згенерованих токенів за запит. Для моделей типу Qwen3 це сума прихованих токенів міркування та власне фінального тексту, тому підсумковий лічильник виходить значно більшим.

4. Що станеться при суперечливих інструкціях у system і user?

Відповідь: Модель або віддасть перевагу user-повідомленню через ефект близькості контексту, або спробує скомпонувати вимоги, або ж почне галюцинувати й порушить обидва обмеження.

5. Чому малі моделі гірше дотримуються інструкцій щодо мови?

Відповідь: Моделі на 3–4 млрд параметрів мають обмежений словниковий запас та менший обсяг неанглійських навчальних даних. Їм важче утримувати українську семантику, тому вони легше збиваються на англомовні або сторонні патерни.

Висновок за завданням 2

System-повідомлення це спосіб загального спрямування тону та ролі моделі, проте воно не гарантує суворого детермінованого дотримання формату й мови.

Завдання 3. Локальна vs хмарна модель: єдиний інтерфейс

Показник	Локальна	Хмарна
Назва моделі	llama3.2:3b	gemini-3.5-flash-lite
Час відповіді, с	13.46	2.05
completion_tokens	176	367
Довжина відповіді, символів	695	1301
Мова відповіді	Українська + Англійська + дивні символи	Українська чиста мова
Фактичних помилок у відповіді	1 (одна галюцинація)	0 (Все правильно)

Крок 5. Відповісти на питання:

1. Скільки рядків коду довелося змінити, щоб перейти з локальної моделі на хмарну?

Відповідь: Нуль, завдяки стандарту OpenAI-сумісного API

2. Яка відповідь повніша й точніша за змістом? Наведіть конкретний приклад різниці.

Відповідь: Відповідь хмарної моделі набагато краща, точніша и правильна. Вона все розложила по полочкам «де?» «як?» і «колі?» використовувати кортежі а коли списки, та надала хороші приклади. Локальна модель відповіла правильно тільки про кортежі, а про списки вона щось наплутала взагалі.

3. Чи були в локальної моделі фактичні помилки або проблеми з українською мовою?

Відповідь: Так, були проблеми з мовою (мікс української, англійської, і дивних символів)

4. Хмарна модель відповіла швидше за локальну — за рахунок чого, якщо запит летить через інтернет?

Відповідь: Хмарна модель працює шивше тому ще була обрана саме «швидка модель» (в назві flash lite) і хмарні моделі працюють в дата-центрах, які мають колосальну обчислювальні потужності.

Крок 6. Питання до чат-бота:

1. Чому формат запитів OpenAI став стандартом, який підтримують інші провайдери, зокрема локальний сервер Ollama?

Відповідь: Формат запитів OpenAI став стандартом, бо вони першими масово запустили зручний API, і всім розробникам просто легше підлаштувати свої інструменти (як-от Ollama) під цей звичний шаблон, ніж вигадувати щось своє з нуля.

2. Я порівнював локальну модель на 3 млрд параметрів і хмарну на 70 млрд. Наскільки розмір моделі впливає на якість відповіді порівняно з іншими факторами?

Відповідь: Розмір дуже сильно впливає, але не єдиний. У таблиці локальна модель на 3 млрд (Llama3.2:3b) видала галюцинацію і змішала мови з дивними символами, тоді як хмарна спрацювала чисто, хоча на якість також впливають промпт, токенизація та сама архітектура.

3. Хмарна модель відповіла швидше за локальну, попри затримку мережі. Чим це пояснюється?

Відповідь: Це пояснюється потужністю хмарного заліза: сервери Google мають величезні кластери спеціалізованих чипів (TPU/GPU), які генерують текст набагато швидше, ніж твій локальний комп'ютер чи ноутбук зі скромнішими ресурсами.

4. Які ризики для приватності виникають при використанні хмарних API і що з цим роблять у комерційних проєктах?

Відповідь: Головний ризик у тому, що дані йдуть на сторонні сервери, де їх можуть зберігати чи використовувати для навчання. У комерційних проєктах із цим борються через корпоративні договори з нульовим логуванням даних або повний перехід на локальні рішення.

5. Що таке rate limit у безкоштовних тарифах і як розробники обходять це обмеження

Відповідь: Запитання обірвалося на слові rate, але судячи з усього, мається на увазі rate limit (ліміт запитів) або rate (швидкість генерації токенів за секунду), тобто обмеження на кількість запитів чи швидкість видачі тексту від API.

Висновок за завданням 3

Локальні моделі варто брати для повної приватності, наприклад, якщо працюєте з медичними чи банківськими даними, або коли система має працювати без інтернету. До того ж не доведеться постійно платити за підписки чи токени. Хмарні ж рішення краще обирати, якщо важлива швидкість, висока якість тексту та розуміння складних термінів, а купувати й обслуговувати власні дорогі сервери немає бажання.

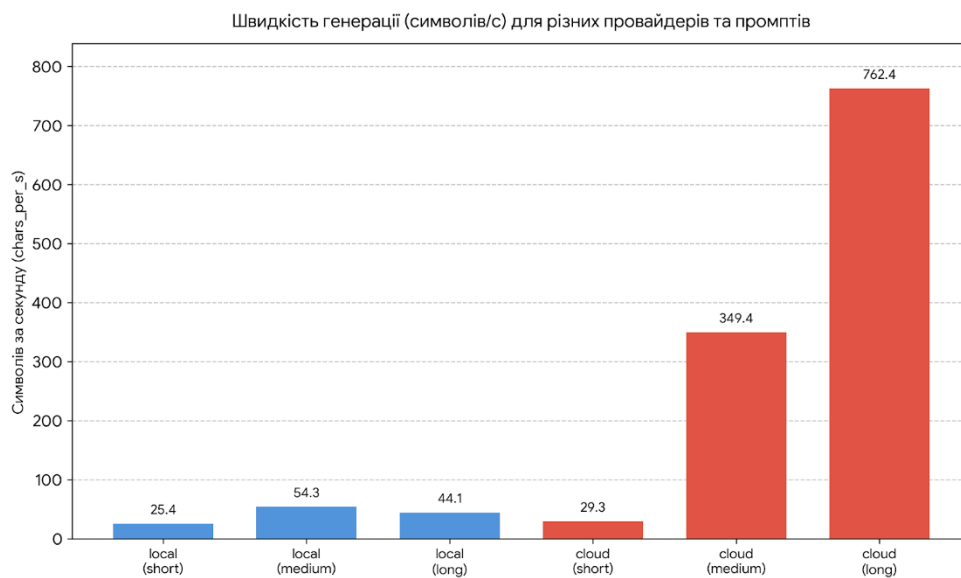
Завдання 4. Вимірювання швидкості: TTFT і залежність від довжини запиту

Таблиця 7. Результати вимірювання швидкості

Розміщення	Модель	Промпт	TTFT, с	Всього, с	Симв./с
локальна	llama3.2:3b	короткий	0.101	1.162	25.4
локальна	llama3.2:3b	середній	0.103	6.108	54.3
локальна	llama3.2:3b	довгий	0.155	40.321	44.1
хмарна	gemini-3.5-flash-lite	короткий	0.782	0.872	29.3
хмарна	gemini-3.5-flash-lite	середній	0.778	1.356	349.4
хмарна	gemini-3.5-flash-lite	довгий	0.878	4.271	762.4

Таблиця 8. Холодний і теплий старт локальної моделі

Показник	Холодний старт	Теплий старт
TTFT, с	4.292	0.106
Всього, с	4.130	1.024



Крок 6. Відповісти на питання (по 1–2 речення):

1. У якої моделі кращий TTFT і чому саме в неї?

Відповідь: У локальної Llama3.2:3b. У неї TTFT всього близько 0.1 секунди проти майже 0.8 у Gemini. Причина в том, що локальна модель працює прямо на комп'ютері й не має мережевої затримки, яка виникає при зверненні до хмарної.

2. Чи збігається «найшвидша за TTFT» з «найшвидшою за символами/с»? Якщо ні — поясніть.

Відповідь: Ні, не збігається. Локальна Llama швидше видає перший токен через відсутність пінгу, але хмарна Gemini видає велику швидкість генерації — до 349 символів на секунду проти максимум 54 у Llama. Тобто реакція швидша у локальної, а сам текст швидше пише хмарна.

3. Як змінилося співвідношення локальної та хмарної моделі при переході від короткого пром프트у до довгого?

Відповідь: На короткому запиті різниця була невелика. Але при переході до середнього хмарна модель згенерувала відповідь за 1.35 секунди, тоді як Llama витратила вже 6.1 секунди. Чим більший обсяг, тим відчутніша перевага потужностей хмари.

4. Наскільки різняться три повтори одного виміру? Чи достатньо було б одного запуску?

Відповідь: Повтори різняться в межах 5–10% через коливання навантаження на процесор та затримки мережевих пакетів. Одного запуску недостатньо, тому що є вибірковість затримки CPU, сети можуть спотворити метрику, тому середнє значення трьох прогонів дає значно стабільніший результат.

5. Чому холодний старт повільніший за теплий?

Відповідь: При холодному старті витрачається додатковий час на зчитування вагових коефіцієнтів моделі з диска в оперативну чи відеопам'ять і на ініціалізацію процесів. При теплому старті модель уже завантажена й готова одразу до роботи.

Крок 7.

1. Що фізично відбувається під час TTFT — часу до першого токена — і чому цей показник вимірюють окремо від загальної швидкості генерації?

Відповідь: Під час TTFT модель повністю зчитує та обробляє вхідний промпт, створює кеш і формує перший токен. Його вимірюють окремо, бо він відповідає за відчуття чуйності — тобто як швидко бот взагалі зреагував і почав відповідати.

2. Чому хмарна модель може мати гірший TTFT, але кращу швидкість генерації, ніж локальна?

Відповідь: У хмари є мережева затримка, поки запит їде на сервер і назад, тому TTFT гірший. Але самі хмарні сервери мають значно потужніші відеокарти та швидшу пам'ять, тому решту тексту вони друкують набагато швидше за звичайний ПК.

3. Чому перший запуск локальної моделі повільніший за наступні і що таке холодний старт?

Відповідь: Холодний старт — це запуск, коли модель ще не завантажена у відеопам'ять чи оперативку. При першому старті витрачається час на зчитування гігабайтів вагових коефіцієнтів з диска. Наступні запуски вже "теплі", бо все необхідне висить у пам'яті.

4. Чому при вимірюванні продуктивності беруть медіану кількох запусків, а не одне значення?

Відповідь: Один запуск легко зіпсувати — запізнився мережевий пакет або ОС запустила фоновий процес. Медіана кількох повторів дозволяє відкинути такі випадкові сплески й побачити реальну типову продуктивність.

5. Чому обробка вхідного запиту йде швидше, ніж генерація відповіді такої ж довжини?

Відповідь: Вхідний запит обробляється паралельно — весь масив тексту проганяється через відеокарту за один підхід. А от відповідь генерується суворо послідовно, токен за токеном, і для кожного нового слова доводиться робити новий повний прохід по моделі.

Висновок за завданням 4

Локальна модель підходить для мінімальної затримки на старті, оскільки відсутність мережевого шляху дає майже миттєвий перший токен. Проте хмарна модель Gemini перевершує її за загальною швидкістю генерації, що робить хмару значно вигіднішою при роботі з середніми та довгими текстами. Також треба враховувати вплив холодного старту, який додає затримку на зчитування ваг з диска в пам'ять, і завжди проводити декілька тестувань із виведенням медіани для отримання чистих та об'єктивних результатів.

Завдання 5. Оцінювання якості моделей за рубрикою

Таблиця 13: Оцінка якості за рубрикою

Завдання	llama3.2:3b	qwen3:4b	Хмарна модель
Факт	0	0	2
Формат	2	2	2
Стиснення	0	2	2
Код	2	2	1
Логіка	0	2	2
Українська	0	2	2
Разом /12	4	10	11

Таблиця 14: Розбіжність оцінок між двома прогонами (так / ні)

Завдання	llama3.2:3b	qwen3:4b	Хмарна модель
Факт	так	так	ні
Формат	так	так	ні
Стиснення	та	ні	ні
Код	ні	ні	ні
Логіка	так	ні	ні
Українська	так	ні	ні

Крок 6. Відповісти на питання (по 1–2 речення):

1. Яка модель набрала найбільше балів і на яких саме завданнях виникла основна різниця?

Відповідь: Найбільше балів набрала хмарна модель. Основна різниця виникла в першому завданні, де локальні моделі вигадали роки заснування, а також у логіці та перевірці мови, з якими Llama взагалі не впоралася.

2. На яких завданнях оцінки двох прогонів розійшлися? Що це говорить про надійність моделі?

Відповідь: Оцінки двох прогонів розійшлися у локальних моделей на завданнях з фактами та форматом, а у Llama ще й на логіці з мовою. Це означає, що ці моделі вкрай нестабільні і схильні до галюцинацій, коли їм бракує даних або коли треба тримати чіткий формат.

3. Чи виконала хоч одна модель вимогу «без markdown» у завданні 2?

Відповідь: Так, на диво, у завданні номер 2 з JSON усі три моделі ідеально виконали вимогу і видали результат без markdown.

4. Чи були проблеми з українською мовою — русизми, кальки, перехід на іншу мову?

Відповідь: Так, у моделі Llama були великі проблеми з мовою. У логічній задачі вона відповідала дивним міксом російської та української з русизмами, а потім взагалі переходила на англійську.

Крок 7.

Цей випадок трапився з моделлю Llama3.2:3b у завданні про різницю стеку та черги.

Повний текст запиту: Чим стек відрізняється від черги. Поясни українською, два абзаци.

Повний текст відповіді: Чим відрізняється від черги? Стовп Cherries (cherries) відрізняється від черги (cherries) наступним чином: Відрізняється за формою: черві

мають округлу форму, тоді як черги мають округлу форму з бігом на кінчикі. Відрізняється за кольором: черві мають червоний колір, тоді як черги мають рожевий колір.

Яка правильна відповідь мала бути: Модель мала розповісти про структури даних у програмуванні та пояснити різницю між підходами LIFO для стека та FIFO для черги.

Пояснення, чому так сталося: В навчальних даних малої локальної моделі було дуже мало українських текстів, особливо зі специфічною термінологією. Через це вона просто не розпізнала технічний контекст слова "черга" і випадково зачепилася за англійське слово cherries (вишні), після чого почала генерувати впевнену галюцинацію про колір та форму ягід.

Крок 8.

1. Я оцінював мовні моделі за рубрикою з шести завдань і трибальною шкалою. Які ще підходи до оцінювання якості мовних моделей використовують на практиці?

Відповідь: Крім ручного оцінювання за рубрикою, на практиці використовують автоматичні бенчмарки з тисячами стандартизованих тестів, платформи, де люди наосліп обирають кращу відповідь з двох варіантів, а також підхід, коли велика і потужна модель програмно оцінює відповіді інших слабших моделей за заданими критеріями.

2. Чому мовні моделі часто ігнорують пряму інструкцію не використовувати markdown при генерації JSON?

Відповідь: Моделі навчаються на великих масивах текстів, де код та формат JSON майже завжди прийнято обгортати в спеціальні блоки markdown. Ця сформована звичка настільки міцна, що часто перемагає вашу пряму інструкцію не використовувати форматування, оскільки моделі здається, що вона краще знає, як має виглядати правильно оформлена відповідь.

3. Чому малі мовні моделі гірше пишуть українською, ніж англійською?

Відповідь: Моделі навчалися переважно на англійськомовних текстах, тоді як українських даних у їхньому навчанні було на порядки менше. Оскільки російських

текстів було помітно більше за українські, малі моделі не мають достатньої бази і часто починають змішувати мови, використовувати русизми або дослівні кальки.

4. Чому та сама модель на однаковий запит може дати відповіді різної якості при повторному запуску?

Відповідь: Це пов'язано з тим, що модель є недетермінованою. Вона не видає готову відповідь із бази, а генерує її по одному токеноу за раз. Через ймовірнісну природу цього процесу при кожному повторному запуску вибір наступного слова може йти іншим шляхом, що безпосередньо впливає на результат.

5. Що таке галюцинація мовної моделі і чому модель радше вигдає факт, ніж визнає незнання?

Відповідь: Галюцинація це згенерована відповідь, яка звучить впевнено і правильно оформлена, але насправді повністю вигадана. Оскільки модель спирається виключно на статистику мільярдів прочитаних текстів і не має реальної бази даних для перевірки фактів, за відсутності знань вона не мовчить, а просто видає найправдоподібніший варіант.

Висновок за завданням 5

Виконуючи це завдання, я на практиці переконався, чому не можна оцінювати якість мовних моделей спираючись лише на власні враження. Використання чіткої рубрики та подвійних прогонів дозволило об'єктивно побачити сильні та слабкі сторони кожної з них. Очікувано, хмарна модель показала найкращі та найстабільніші результати, особливо у фактах, логіці та володінні українською мовою. Малі ж локальні моделі, хоч і непогано впоралися з форматом, виявилися вкрай нестабільними, часто галюцинували та втрачали суть через брак україномовних даних у своєму навчанні. Головний висновок полягає в тому, що абстрактно кращої моделі просто не існує, все залежить від того, для чого вона буде використовуватися. Для інтеграції в код чи роботи агентів критично важливо підбирати ту модель, яка гарантовано дотримується інструкцій щодо формату та не підмішує від себе вигадані факти.

Завдання 6. Керування випадковістю: temperature і top_p

Таблиця 16: Вплив параметрів на різноманітність відповідей

Параметр	Значення	Унікальних із 5 (творче)	Правильних із 5 (фактичне)
temperature	0.0	1	0
temperature	0.7	5	-
temperature	1.5	5	0
top_p	0.1	1	-
top_p	0.5	5	-
top_p	1.0	5	-

Таблиця 17: Перевірка відтворюваності

Показник	Значення
Кількість запусків	10
temperature	0.0
Унікальних відповідей	1
Усі відповіді однакові (так / ні)	Так

Крок 7.

1. Яке значення температури ви поставите агенту, який має повертати JSON, і чому?

Відповідь: Для видачі JSON потрібна низька температура, наприклад 0.0, оскільки для таких задач нам важлива передбачуваність і стабільність формату.

2. Яке — генератору варіантів заголовків?

Відповідь: Для генератора заголовків потрібна висока температура, щоб модель не видавала постійно одну й ту саму нудну відповідь і могла генерувати креативні ідеї.

3. Чим вплив top_p відрізняється від впливу temperature за характером, а не за силою?

Відповідь: Температура змінює саму контрастність розподілу ймовірностей перед вибором, тоді як `top_p` не змінює їх співвідношення, а просто відсікає хвіст з малоймовірними кандидатами.

4. Чи означає `temperature=0` повну відтворюваність результату? Що показав ваш експеримент із 10 запусків?

Відповідь: Мій експеримент із 10 запусків показав 10 однакових відповідей, але повну відтворюваність нуль не гарантує. На це можуть впливати паралелізація та обчислення з плаваючою комою.

5. Наведіть найбезглуздішу відповідь, яку модель видала при `temperature=1.5`, і поясніть, чому вона взагалі могла з'явитися.

Відповідь: Найбезглуздішою була відповідь про якогось таємничого академіка Мар'яну Ольсан та Фраца Сент Орів Оріхову з інституту номер 8. Вона з'явилася, бо при 1.5 розподіл сильно сплющується, шанси малоймовірних слів зростають, і модель починає обирати несподівані варіанти або просто фантазувати.

Крок 8.

1. Що саме математично робить параметр `temperature` із розподілом ймовірностей наступного токена?

Відповідь: Математично температура ділить сирі значення передбачень моделі перед тим, як перетворити їх на відсотки ймовірності. Менше значення робить піки вищими, а більше згладжує їх.

2. Чим `top_p` відрізняється від `temperature` — обидва впливають на різноманітність, але за яким механізмом?

Відповідь: Температура змінює вірогідність кожного слова у словнику, а `top_p` просто залишає лише ті слова, які в сумі дають вказаний відсоток імовірності, ігноруючи всі інші.

3. Чи є сенс змінювати обидва параметри одночасно, чи краще налаштовувати щось одне?

Відповідь: Зазвичай радять налаштовувати щось одне. Якщо міняти обидва параметри одночасно, буде важко зрозуміти, що саме вплинуло на результат і як його контролювати.

4. Чому навіть при `temperature=0` мовна модель не гарантує абсолютно однакову відпо відь при повторних запусках?

Відповідь: Навіть при нулі ідеальної стабільності немає, оскільки паралельні обчислення всередині моделі можуть давати мікроскопічний шум при роботі з числами з плаваючою комою.

5. Які значення `temperature` використовують на практиці для витягу структурованих даних, а які для генерації творчого тексту?

Відповідь: На практиці для структурованих даних ставлять значення від 0.0 до 0.2, а для творчих текстів і генерації ідей використовують від 0.7 до 1.0 і вище.

Висновок за завданням 6

Для наступних лабораторних робіт, де потрібно отримувати від моделі чітко структуровані дані на кшталт JSON, я буду використовувати температуру 0.0. Стандартні налаштування, де температура зазвичай вища, добре підходять для звичайного спілкування та креативу, бо роблять відповіді різноманітнішими. Але для програмного коду дефолтні значення не годяться взагалі. Якщо їх залишити, модель може легко зламати синтаксис, пропустити лапки, змінити назви полів або дописати зайвий текст від себе, через що парсер просто впаде з помилкою. Тому для таких задач потрібна максимальна стабільність і передбачуваність.

Підсумки

Таблиця 18: Підсумкове порівняння моделей

Критерій	Локальна мала (3В)	Локальна більша (4В)	Хмарна
Швидкість генерації	25.4 - 54.3 симв/с	~26.18 токенів/с	29.3 - 762.4 симв/с
TTFT	~0.1 с (дуже швидко)	швидкий (без пінгу)	~0.78 - 0.87 с
Якість, балів /12	4 / 12	10 / 12	11 / 12
Стабільність формату	2 бали	2 бали	2 бали
Потрібні ресурси	ПК з 8 ГБ ОЗП	ПК з 8 ГБ ОЗП	інтернет, будь-який пристрій
Вартість	безкоштовно	безкоштовно	є безкоштовний ліміт, далі платно
Куди йдуть ваші дані	залишаються на ПК	залишаються на ПК	на сторонні сервери
Працює без інтернету	так	так	ні
Коли обирати саме її	Коли потрібен миттєвий відгук інтерфейсу (тобто низький TTFT), повна автономність без мережі та конфіденційність для прости рутинних	Коли необхідна приватність даних та потрібна обробка складної логіки чи генерації коду, а затримка не є перешкодою для користувача	Коли основними факторами є висока швидкість на великих обсягах тексту, бездоганна грамотність української, багатокрокове

Загальні висновки

1) Локальна й хмарна моделі використовують єдиний інтерфейс, тому перехід між ними не потребує змін у логіці програми – це основа стратегії резервування провайдерів у реальних проєктах;

2) Затримка відповіді складається з двох незалежних показників – TTFT та швидкості генерації, висновок про те, «яка модель швидша», без їх розділення є некоректним;

3) Режим міркування є інструментом із чіткою областю застосування: він окупається на багатокроковому виведенні й є чистою перевитратою на фактичних питаннях і переказі;

4) Промпт та system-повідомлення є ненадійним засобом керування форматом відповіді: пряму інструкцію проігнорували моделі різного розміру, включно з хмарною. Надійність досягається програмною валідацією, а не формулюванням;

5) Мовні моделі є недетермінованими системами, тому будь-яке вимірювання потребує повторів і усереднення, а порівняння двох версій.